

Personal Finance Dashboard

Projektantrag

Roman Lysser · 02.09.2026

1. Kontext

Private Haushaltsfinanzen sind über Kontoauszüge, Kreditkartenabrechnungen und Tabellenkalkulationen verstreut, eine verlässliche Gesamtsicht fehlt. Das Personal Finance Dashboard bündelt Einnahmen und Ausgaben eines Haushalts an einem Ort, führt sie je Konto, ordnet jede Buchung einer Kategorie zu und stellt darauf Monatsbudgets und Auswertungen. Eine Bankanbindung findet bewusst nicht statt: Buchungen werden von Hand erfasst, der Kontostand ergibt sich aus Anfangsbestand und erfassten Buchungen. Zielgruppe sind Haushalte, die ihre Finanzen ohne Bankanbindung und ohne Weitergabe von Daten an Dritte selbst führen wollen.

2. User Stories

Priorisierung nach MoSCoW: **Must have** ist für ein nutzbares Produkt zwingend, **Should have** ist wichtig, aber verschiebbar, **Could have** ist Zusatznutzen bei freier Kapazität, **Won't have** ist für diese Iteration bewusst ausgeschlossen. Die Reihenfolge innerhalb einer Stufe entspricht der geplanten Umsetzungsreihenfolge.

2.1. Must have

ID	User Story
M1	Als Nutzer möchte ich die Anwendung über eine simulierte Anmeldung mit einem festen Haushalt betreten, damit die gesamte Fachlichkeit nutzbar ist, bevor die echte Authentifizierung existiert.
M2	Als Nutzer möchte ich, dass jedes Konto, jede Buchung und jede Kategorie genau einem Haushalt gehört, damit die Datengrenze von Beginn an steht und die echte Anmeldung später nur noch bestimmt, welcher Haushalt gilt.
M3	Als Nutzer möchte ich meine Daten dauerhaft gespeichert wissen, damit sie nach dem Neuladen der Anwendung erhalten bleiben.
M4	Als Nutzer möchte ich beim Anlegen des Haushalts eine sinnvolle Kategorienliste vorfinden und diese erweitern und umbenennen können, damit ich sofort erfassen kann und die Struktur trotzdem zu mir passt.
M5	Als Nutzer möchte ich auch ohne Kategoriewahl erfassen können, wobei die Buchung einer reservierten Sammelkategorie zufällt, damit die Schnellerfassung nie blockiert.

ID	User Story
M6	Als Nutzer möchte ich beim Löschen einer Kategorie wählen, ob ihre Buchungen in eine andere Kategorie überführt oder der reservierten Sammelkategorie zugeordnet werden, damit keine Buchung ohne Zuordnung zurückbleibt.
M7	Als Nutzer möchte ich ein Konto mit Name, Kontoart und Anfangsbestand anlegen und umbenennen, damit ich meine Buchungen dem richtigen Konto zuordnen kann.
M8	Als Nutzer möchte ich, dass sich der Kontostand ausschliesslich aus Anfangsbestand und erfassten Buchungen ergibt, damit die angezeigte Zahl immer zu meiner Historie passt.
M9	Als Nutzer möchte ich eine Ausgabe mit Betrag und Kategorie erfassen können, wobei Datum und Konto sinnvoll vorbelegt sind, damit die Erfassung unterwegs wenige Sekunden dauert.
M10	Als Nutzer möchte ich eine Einnahme erfassen, die als Zufluss zählt und keinen Ausgabencharakter hat, damit ich meinen Zufluss den Ausgaben gegenüberstellen kann.
M11	Als Nutzer möchte ich jede Buchung nachträglich ändern und löschen können, wobei sich der Kontostand sofort mitkorrigiert, damit Tippfehler folgenlos bleiben.
M12	Als Nutzer möchte ich alle Buchungen des Haushalts chronologisch, seitenweise und gefiltert nach Konto und Kategorie durchsehen, damit ich einzelne Posten wiederfinde.
M13	Als Nutzer möchte ich eine Startansicht mit Kontoständen, Haushaltssumme, Monatscashflow und den letzten zehn Buchungen sehen, damit ich meine Lage auf einen Blick erfasse.
M14	Als Nutzer möchte ich direkt aus der Startansicht heraus eine Buchung erfassen, ohne die Ansicht zu verlassen, damit der häufigste Vorgang der kürzeste ist.
M15	Als Nutzer möchte ich die Anwendung auf Smartphone, Tablet und Desktop gleichwertig bedienen können, damit ich unterwegs erfasse und am grossen Bildschirm auswerte.
M16	Als Nutzer möchte ich ein Konto löschen können, solange keine Buchung darauf erfasst ist, und es andernfalls archivieren, damit auch Konten vollständig verwaltbar sind, ohne dass Historie verloren geht.
M17	Als Nutzer möchte ich den Ausgabenanteil je Kategorie für einen wählbaren Monat als Diagramm sehen, damit meine Daten neben Liste und Kennzahlen auch in grafischer Form vorliegen.
M18	Als Entwickler möchte ich die Fachlichkeit durch automatisierte Tests abgesichert wissen — Unit-Tests der Berechnungslogik, Integrationstests gegen die API, E2E-Tests der Kernabläufe —, damit Änderungen nicht unbemerkt Verhalten brechen.

ID	User Story
M19	Als Entwickler möchte ich einen Lighthouse-Score von mindestens 90 im Durchschnitt für Mobile und Desktop erreichen, damit die Anwendung nachweislich schnell und zugänglich ist.
M20	Als Betreiber möchte ich das Prod-Bundle mit einem einzigen Befehl (docker compose up) reproduzierbar starten, damit die Anwendung ohne Zusatzwissen lauffähig ist.
M21	Als Entwickler möchte ich eine feature-orientierte Struktur mit durchgängiger Typisierung und einheitlicher Formatierung, damit der Code lesbar bleibt und sich um die Could-Have-Fachlichkeit erweitern lässt.

2.2. Should have

ID	User Story
S1	Als Nutzer möchte ich je Kategorie und Kalendermonat eine Obergrenze anlegen, ändern und wieder entfernen können, damit ich meine Ausgabenabsicht verbindlich mache und laufend anpassen kann.
S2	Als Nutzer möchte ich alle Budgets eines wählbaren Monats gesammelt sehen — je Kategorie mit Limit, Verbrauch und Rest —, damit ich meine Budgetplanung an einem Ort pflege.
S3	Als Nutzer möchte ich zwischen „kein Limit“ und „Limit 0“ unterscheiden können, damit ich eine Kategorie entweder bewusst nicht budgetiere oder bewusst auf null setze.
S4	Als Nutzer möchte ich, dass ein neuer Monat die Limiten des Vormonats übernimmt und ich sie dort ändern kann, ohne den Vormonat zu verändern, damit ich nicht jeden Monat alles neu erfasse.
S5	Als Nutzer möchte ich den Budgetverbrauch je Kategorie sehen und bei Überschreitung den Überschreibungsbetrag genannt bekommen, damit ich vor Monatsende gegensteuern kann.
S6	Als Nutzer möchte ich eine Safe-to-Spend-Kennzahl für den laufenden Monat sehen, damit ich eine einzige ehrliche Zahl statt vieler Teilbudgets im Kopf habe.
S7	Als Nutzer möchte ich Budgetfortschritt und Safe-to-Spend in der Startansicht sehen, damit die zentrale Ansicht die Budgetlage mit abdeckt.

2.3. Could have

ID	User Story
C1	Als Nutzer möchte ich mich mit E-Mail und Passwort registrieren und anmelden, wobei mit der Registrierung mein Haushalt entsteht und ich dessen Eigentümer bin, damit die simulierte Anmeldung durch echten Zugriffsschutz ersetzt wird.

ID	User Story
C2	Als Nutzer möchte ich den realen Kontostand eintragen und die Anwendung die Differenz als Saldokorrektur verbuchen lassen, damit App und Bank wieder übereinstimmen, ohne dass ich rechne.
C3	Als Nutzer möchte ich, dass eine Saldokorrektur standardmässig kein Budget belastet, ich ihr aber nachträglich eine Kategorie zuweisen kann, damit vergessene Ausgaben meine Budgetzahlen nicht verfälschen.
C4	Als Nutzer möchte ich Einnahmen und Ausgaben der letzten zwölf Monate im Verlauf sehen, damit ich beurteilen kann, ob ich über meine Verhältnisse lebe.
C5	Als Nutzer möchte ich die Auswertung nach Kategorie und Konto filtern, wobei der Filter beide Darstellungen zugleich einschränkt, damit ich einer Frage gezielt nachgehen kann.
C6	Als Nutzer möchte ich eine wiederkehrende Buchung mit Betrag, Kategorie, Konto, Startdatum und Intervall anlegen — täglich, wöchentlich, monatlich, quartalsweise, halbjährlich oder jährlich —, damit ich Fixkosten wie Miete oder Abonnements nicht jeden Monat neu erfasse.
C7	Als Nutzer möchte ich bei wöchentlichem Intervall den Wochentag und bei monatlichem, quartalsweisem, halbjährlichem und jährlichem Intervall den Tag im Monat festlegen, damit die Buchung am tatsächlichen Fälligkeitstag entsteht.
C8	Als Nutzer möchte ich, dass eine fällige wiederkehrende Buchung um Mitternacht des Fälligkeitstags automatisch als Buchung entsteht, damit Kontostand und Monatssummen ohne mein Zutun stimmen.
C9	Als Nutzer möchte ich je wiederkehrende Buchung festlegen, ob der Betrag fest ist oder schwankt, damit schwankende Beträge zur Prüfung gekennzeichnet werden und feste Beträge stillschweigend gebucht werden.
C10	Als Nutzer möchte ich, dass ein Tag im Monat, den es in einem kürzeren Monat nicht gibt, auf dessen letzten Tag fällt, damit der 31. auch im Februar zu genau einer Buchung führt.
C11	Als Nutzer möchte ich wiederkehrende Buchungen auflisten, ändern, pausieren und löschen, wobei bereits entstandene Buchungen unverändert bleiben, damit ich Fixkosten pflegen kann, ohne meine Historie zu verfälschen.
C12	Als Eigentümer möchte ich Mitglieder entfernen, die Eigentümerschaft übertragen und den Haushalt löschen können, damit klar geregelt ist, wer über die geteilten Daten bestimmt.
C13	Als Nutzer möchte ich beim Erfassen einen Kategorievorschlag auf Basis des zuletzt für diesen Zahlungsempfänger verwendeten Eintrags erhalten, damit die Erfassung schneller geht.

2.4. Won't have (in dieser Iteration)

ID	Ausschluss und Begründung
W1	Automatischer Bankimport über PSD2/Open Banking — hoher regulatorischer und technischer Aufwand; die Saldokorrektur deckt den Abgleich fachlich ab.
W2	Umbuchungen zwischen eigenen Konten — verlangen Ausschluss aus Budgets und Cashflow und damit eine zweite Buchungsart.
W3	Kreditkarten- und Anlagekonten — die Vorzeichenlogik ist im Datenmodell vorbereitet, die Fachlichkeit bleibt dieser Iteration erspart.
W4	Einladen weiterer Mitglieder in den Haushalt — die Rollen und die Datengrenze sind spezifiziert, der Einladungsablauf folgt in einer späteren Iteration.
W5	Private, nur für ein Mitglied sichtbare Konten — der Haushalt teilt in dieser Iteration eine gemeinsame Sicht.
W6	Mehrwährungsfähigkeit inklusive Wechselkursen — eine Währung je Haushalt genügt für den Kernnutzen.
W7	Budgetübertrag in den Folgemonat und Umschlagmethode — jeder Kalendermonat wird bewusst unabhängig bewertet.
W8	Budgets je Mitglied sowie abweichende Periodengrenzen (z. B. Zahltagszyklus) — der Kalendermonat und ein gemeinsamer Budgetsatz halten die Auswertung eindeutig.
W9	Hinweis beim Erreichen eines Budgetanteils (z. B. 80 Prozent) — Schwellenmeldungen werden zu Rauschen; Safe-to-Spend leistet dasselbe ohne Benachrichtigung.
W10	CSV-Import und -Export sowie Volltextsuche — Filter nach Konto und Kategorie decken das Wiederfinden ab.
W11	Sparziele und Vermögensverlauf — ohne Umbuchungen und Anlagekonten nicht sinnvoll darstellbar.
W12	Native Mobile Apps für iOS und Android — die Weboberfläche wird mobile-first und responsiv ausgeliefert.
W13	Prognose künftiger Ausgaben mittels Machine Learning — ohne belastbare Datenbasis nicht sinnvoll.

3. Angedachter Technologie-Stack

Bun dient in Frontend und Backend als Laufzeit, Paketmanager und Testrunner — ein Werkzeug für das gesamte Repository.

3.1. Frontend

Baustein	Wahl und Begründung
Framework	Angular 22 mit Standalone Components und Signals — typsichere Struktur, klare Trennung von Zustand und Darstellung, Lazy Loading pro Feature-Route für den Lighthouse-Zielwert.
Laufzeit/Tooling	Bun als Paketmanager und Script-Runner.
Sprache	TypeScript — durchgängige Typisierung von der API bis zur Ansicht.
Styling	Tailwind CSS 4 über PostCSS — mobile-first: Basis-Layout für schmale Viewports, Breakpoints erweitern für Tablet und Desktop.
Diagramme	Zwei Kandidaten: TanStack (headless, volle Kontrolle über Markup und damit gute Integration ins eigene Tailwind-Design) und Chart.js (etabliert, canvas-basiert, geringer Integrationsaufwand). Entscheid nach einem kurzen Spike; Kriterium ist die Reife der Angular-Anbindung, da TanStack-Adapter primär für React gepflegt werden. Fallback: ngx-charts.
API-Client	hey-api (@hey-api/openapi-ts) generiert typsicheren Client aus dem vom Backend bereitgestellten OpenAPI-Schema — kein manuell gepflegtes DTO-Duplikat zwischen Frontend und Backend.
Authentifizierung	better-auth Client — Session-Handling gegen die Backend-Auth-Endpunkte.
Tests	Vitest mit jsdom für Komponenten- und Logiktests; Playwright für E2E-Abläufe (Konto anlegen, Buchung erfassen, Buchung korrigieren, Startansicht prüfen, Budget setzen und überschreiten).

3.2. Backend

Baustein	Wahl und Begründung
Framework	NestJS auf Express-Plattform — modulare, feature-orientierte Struktur mit Dependency Injection; Express als bewährte, breit unterstützte HTTP-Basis.
Laufzeit/Tooling	Bun als Paketmanager und Script-Runner.
Sprache	TypeScript — durchgängige Typisierung von der API bis zur Datenbank.
Datenbank/ORM	SQLite über libSQL-Client, angesprochen mit Drizzle ORM — dateibasiert und ohne Zusatzinfrastruktur lauffähig, typsicheres Schema und Migrationen via Drizzle Kit.

Baustein	Wahl und Begründung
API-Dokumentation	@nestjs/swagger erzeugt aus Controllern und DTOs eine OpenAPI-Spezifikation, die dem Frontend als Grundlage für die hey-api-Client-Generierung dient.
Validierung	class-validator und class-transformer — deklarative DTO-Validierung an der API-Grenze.
Authentifizierung	better-auth mit Drizzle-Adapter — Session- und Credential-Handling serverseitig, Schema im selben Drizzle-Schema geführt wie die Fachdaten.
Tests	Vitest für Unit-Tests der Fachlogik; separater E2E-Lauf (vitest.config.e2e.ts) gegen die laufende API.